Iks-oks sa gravitacijom

1. Uvod

U ovom projektu je urađena verzija igre *Iks-oks* (engl. *Tic-Tac-Toe*) na tabli 5x5, ali sa dodatkom „gravitacije“. To znači da se potez ne može odigrati bilo gde, već samo u najnižem slobodnom polju određene kolone – slično kao u igri *Connect Four*.
Pored igre za dva igrača, urađena je i verzija sa **veštačkom inteligencijom (AI)** koja koristi **Minimax algoritam** sa **alpha-beta odsecanjem** za donošenje odluka.

---

2. Struktura programa

Projekat je podeljen na više klasa, što olakšava organizaciju koda i kasnije testiranje:

- **GameLogic** – osnovna klasa sa zajedničkim metodama i logikom igre.
- **GameLogic5x5** – glavna logika igre za tablu 5x5, uključuje Minimax pretragu i funkciju evaluacije.
- **FourInLine5x5** – klasa koja sadrži grafički deo (JavaFX), detekciju klikova i prikaz poteza.

Ideja je bila da se logika i izgled igre odvoje, što omogućava lakšu izmenu jednog dela bez uticaja na drugi.

---

3. Minimax algoritam i ograničena pretraga

AI donosi odluke koristeći **Minimax algoritam**, koji simulira sve moguće poteze i procenjuje koji vodi do najboljeg rezultata.
Kako bi se ubrzalo izvođenje, dodato je **alpha-beta odsecanje**, koje preskače grane stabla za koje se zna da neće uticati na krajnji rezultat.

U praksi, pretraga cele table 5x5 je nemoguća jer broj mogućih stanja veoma brzo raste. Zato sam uveo ograničenje dubine pretrage (`MAX_DEPTH = 6`) i funkciju evaluacije koja „nagađa“ koliko je pozicija dobra bez potrebe da se ide do kraja igre.

Takođe postoji promenljiva **testFullSearch** koja, ako se uključi, pokušava da pretraži celo stablo — što je korisno za testiranje performansi, ali u praksi dovodi do velikih kašnjenja.

---

4. Funkcija evaluacije

Pošto nije moguće pretražiti sve kombinacije, funkcija evaluacije procenjuje trenutno stanje na tabli.
Za svako polje se proverava koliko potencijalnih linija (horizontalno, vertikalno i dijagonalno) može da se napravi.

Isprobane su različite varijante funkcije evaluacije:

- **Brojanje potencijalnih linija** – svako polje daje bodove u zavisnosti od broja istih simbola u nizu.
- **Ponderisane vrednosti** – davale su tačnije rezultate, ali su bile sporije.

Najbolje rezultate i najstabilnije ponašanje davao je **jednostavni model brojanja potencijala**, pa je on ostavljen kao glavna verzija.

---

5. Testiranje performansi

Da bih proverio koliko je algoritam efikasan, dodao sam merenje **vremena izvršavanja** i **korišćenja memorije** za svaki AI potez.

Merenje vremena je rađeno pomoću `System.nanoTime()`, a memorija se proveravala pomoću `Runtime.getRuntime()`.
Rezultati su pokazali sledeće:

- Kada je **testFullSearch = true**, vreme izvršavanja je veoma dugo (čak i više sekundi), a program postaje spor.
- Kada se koristi **ograničena dubina pretrage**, potezi se računaju trenutno (uglavnom ispod 100 ms), a memorija se koristi u prihvatljivim granicama.

Na osnovu toga, može se reći da **puna pretraga nije izvodljiva** na većim tablama, dok kombinacija ograničene dubine i dobre evaluacije daje odličan balans između brzine i kvaliteta poteza.

---

6. Zaključak

U projektu sam implementirao igru *Iks-oks sa gravitacijom* sa protivnikom koji koristi Minimax algoritam.
Kroz testiranje sam uvideo da je potpuna pretraga nemoguća jer traje predugo, pa je uvedeno ograničenje dubine i funkcija evaluacije.
Ovaj pristup omogućava da AI igra „razumno pametno“, ali da igra i dalje ostane brza i igriva.

Moguća poboljšanja:

- prilagođavanje dubine pretrage u zavisnosti od situacije u igri,
- brža funkcija evaluacije sa boljim prepoznavanjem pobedničkih kombinacija,
- dodavanje različitih nivoa težine AI protivnika.

---

7. Literatura

1. Russell, S. & Norvig, P. – *Artificial Intelligence: A Modern Approach*, Pearson, 2021.
2. Wikipedia: *Minimax algorithm with alpha-beta pruning*
3. Materijali sa predavanja i vežbi.